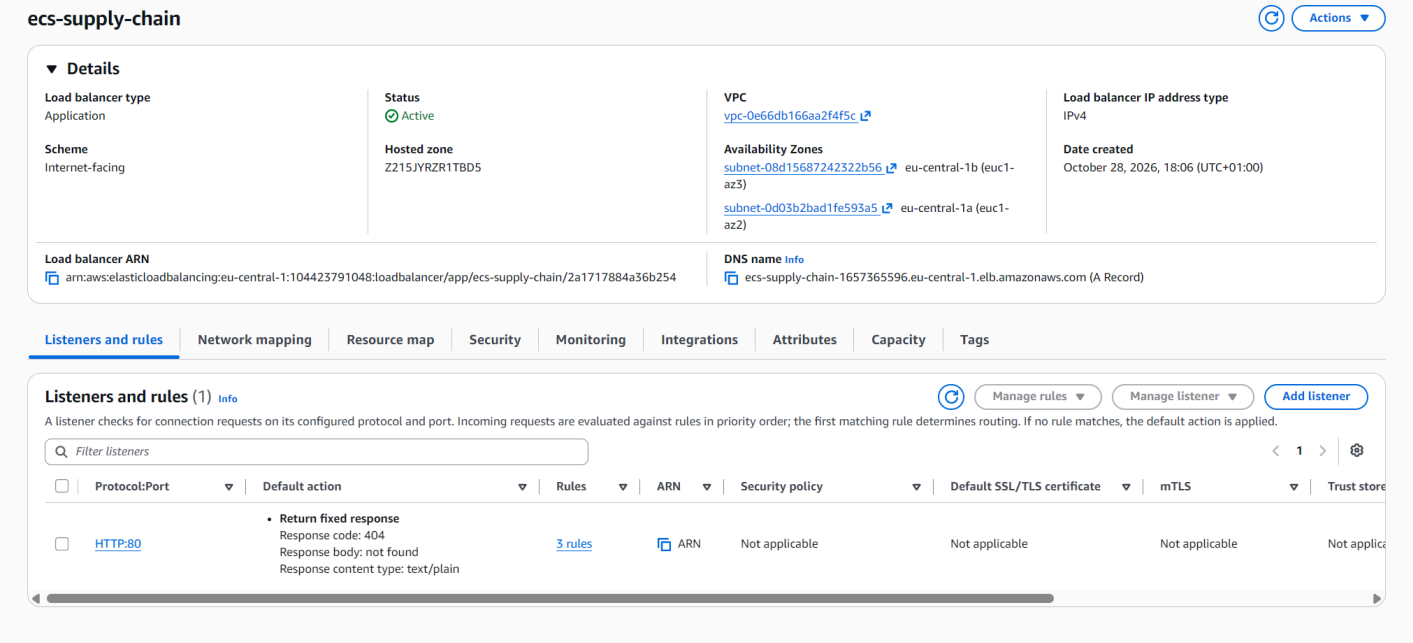
Szkic dokumentu

Hasła i fakty do rozwinięcia, listingi i zrzuty do wklejenia. Diagramy Mermaid do wyrenderowania są w architecture.md, przebieg demo w presentation.md.

2. Architektura

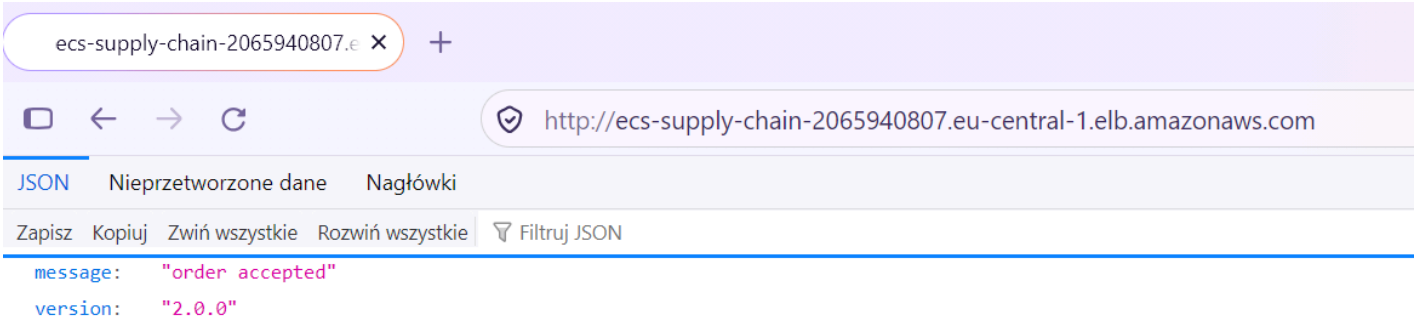
- Diagramy z `architecture.md`: infrastruktura AWS, pipeline `app.yml`, pipeline `infra.yml`, sekwencja blue/green.
- Region `eu-central-1`, jedno środowisko `demo`, całość z jednego `terraform apply` / `destroy`.
- Ruch: internet -> ALB (HTTP 80) -> reguła produkcyjna -> target group blue/green -> zadania Fargate (port 8000).
- Bez NAT Gateway (koszt), zadania w podsieciach publicznych z publicznym IP, SG zadań przyjmuje ruch tylko z SG ALB.



3. Aplikacja

- Python 3.12, FastAPI, uvicorn, 3 endpointy:

Endpoint	Odpowiedź	Użycie
`GET /health`	`{"status":"ok"}`	healthcheck ALB
`GET /version`	wersja i SHA commita	`loadgen.sh`, widać zmianę wersji
`GET /`	prosta odpowiedź biznesowa	ruch testowy



- Zmienne środowiskowe (wpisywane w obraz przez `--build-arg`, nie przez task definition):

Zmienna	Wartości	Po co
`APP_VERSION`	z `app/VERSION`, np. `2.0.0`, przy demo z dopiskiem `-delayed`	widać w `/version`, która wersja obsługuje ruch
`GIT_SHA`	SHA commita	powiązanie obrazu z kodem
`FAILURE_MODE`	`none` / `health` / `delayed`	tryby awarii do demonstracji, wybierane w Run workflow
`FAILURE_DELAY_SECONDS`	domyślnie 90	po ilu sekundach od startu `delayed` zaczyna zwracać 500

- Tryby awarii:
  - `none`: działanie normalne,
  - `health`: `/health` zwraca 500, nowa wersja nie przechodzi healthchecków ALB, nie dostaje ruchu,
  - `delayed`: `/health` OK, ale `/` i `/version` zwracają 500 po 90 s, nowa wersja przejmuje ruch i psuje się w trakcie bake time.
- Dlaczego flaga, a nie osobny zepsuty kod: obraz przechodzi testy i skan, podpis jest ważny, awaria ujawnia się dopiero w środowisku (realistyczny scenariusz).
- Testy pytest: 4 testy, wszystkie tryby.

List. Tryby awarii (`app/main.py`, fragment)

```
def delayed_failure_active() -> bool:
    delay = int(os.getenv("FAILURE_DELAY_SECONDS", "90"))
    return failure_mode() == "delayed" and time.monotonic() - STARTED_AT >= delay


@app.get("/health")
def health():
    if failure_mode() == "health":
        return error()
    return {"status": "ok"}
```

- Dockerfile:
  - multi-stage (etap build z zależnościami, etap końcowy tylko runtime),
  - obraz bazowy przypięty po digeście,
  - użytkownik non-root z UID 10001,
  - w task definition `readonlyRootFilesystem = true`,
  - bez `HEALTHCHECK` (robi go ALB).

List. `app/Dockerfile`

```
FROM python:3.12-slim@sha256:12316debbfb86890f6b2d825820612741ba527ad5a44e3902759141d572bfe8f AS build
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim@sha256:12316debbfb86890f6b2d825820612741ba527ad5a44e3902759141d572bfe8f
ARG APP_VERSION=dev
ARG GIT_SHA=unknown
ARG FAILURE_MODE=none
ENV APP_VERSION=${APP_VERSION} \
    GIT_SHA=${GIT_SHA} \
    FAILURE_MODE=${FAILURE_MODE} \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin app
COPY --from=build /install /usr/local
WORKDIR /app
COPY main.py .
USER 10001
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

4. Infrastruktura (Terraform)

- Jeden root module w `terraform/`, pliki według obszaru:

Plik	Zawartość
`backend.tf`	stan w S3, natywna blokada `use_lockfile`
`versions.tf`	Terraform >= 1.10, provider AWS `~> 6.6` (natywny blue/green od 6.4), `default_tags`
`network.tf`	VPC, 2 podsieci publiczne w 2 AZ, IGW, bez NAT
`ecr.tf`	repozytorium obrazów
`alb.tf`	ALB, listener HTTP 80, target groupy blue i green, reguła produkcyjna i testowa
`ecs.tf`	klaster (Container Insights), task definition, usługa blue/green, SG zadań
`monitoring.tf`	2 alarmy CloudWatch na poziomie ALB
`iam.tf`	role OIDC dla GitHub Actions, rola zadań, rola ECS do blue/green

- Ręcznie (raz, w konsoli): bucket na stan i provider OIDC GitHuba.

List. Usługa ECS z natywnym blue/green (`terraform/ecs.tf`, fragment)

```
deployment_configuration {
  strategy             = "BLUE_GREEN"
  bake_time_in_minutes = 5
}

deployment_circuit_breaker {
  enable   = true
  rollback = true
}

alarms {
  enable   = true
  rollback = true
  alarm_names = [
    aws_cloudwatch_metric_alarm.target_5xx.alarm_name,
    aws_cloudwatch_metric_alarm.response_time.alarm_name,
  ]
}

lifecycle {
  ignore_changes = [task_definition, desired_count, load_balancer]
}
```

- `ignore_changes`: obrazem zarządza pipeline aplikacji, nie Terraform; ECS zamienia blue i green po każdym wdrożeniu.
- `load_balancer.advanced_configuration`: alternatywny target group, reguła produkcyjna i testowa, rola ECS do przełączania wag.

List. Alarm 5xx (`terraform/monitoring.tf`, fragment)

```
metric_name         = "HTTPCode_Target_5XX_Count"
statistic           = "Sum"
period              = 60
evaluation_periods  = 3
datapoints_to_alarm = 2
threshold           = var.alarm_5xx_threshold
```

- Wymiar `LoadBalancer`: alarm obejmuje oba target groupy.
- Czas reakcji około 3 min (2 z 3 punktów po 60 s + opóźnienie metryk ALB), bake time 5 min, więc alarm zdąży przed końcem bake time.

Środki bezpieczeństwa

- **ECR**: tagi `IMMUTABLE` (nie da się nadpisać obrazu pod tym samym tagiem), `scan_on_push`, szyfrowanie KMS, lifecycle 20 obrazów.

List. `terraform/ecr.tf` (fragment)

```
resource "aws_ecr_repository" "app" {
  name                 = local.name
  image_tag_mutability = "IMMUTABLE"
  force_delete         = true

  image_scanning_configuration {
    scan_on_push = true
  }

  encryption_configuration {
    encryption_type = "KMS"
  }
}
```

- **OIDC zamiast kluczy**: GitHub Actions dostaje krótkotrwałe poświadczenia, w repo i w GitHubie brak kluczy AWS.
- **3 role z warunkiem na `sub`**:

Rola	Kto może przyjąć	Uprawnienia
`gh-plan`	PR	ReadOnlyAccess + zapis tylko pliku blokady stanu
`gh-infra-apply`	job w środowisku `infra` (tylko `main`)	zasoby projektu, IAM tylko na role `ecs-supply-chain-*`
`gh-app-deploy`	gałąź `main`	push do 1 repo ECR, 1 usługa ECS, `iam:PassRole` tylko na rolę zadań

- Nowy format `sub` (repozytoria od 15.07.2026): niezmienne ID właściciela i repo, ochrona przed przejęciem nazwy repo.

List. Warunek zaufania ról (`terraform/iam.tf`, fragment)

```
  for_each = {
    plan        = "repo:${var.github_oidc_repo}:pull_request"
    infra-apply = "repo:${var.github_oidc_repo}:environment:infra"
    app-deploy  = "repo:${var.github_oidc_repo}:ref:refs/heads/main"
  }
```

- **Sieć**: SG ALB: wejście tylko 80; SG zadań: wejście tylko z SG ALB na 8000, wyjście tylko 443.
- **Skan konfiguracji Trivy**: blokuje HIGH/CRITICAL, wyjątki tylko w `.trivyignore.yaml` z uzasadnieniem i ograniczone do pliku.

List. `.trivyignore.yaml` (fragment)

```
- id: AWS-0104
  paths:
    - "ecs.tf"
  statement: Tasks run in public subnets without NAT Gateway or VPC endpoints (cost), so they need HTTPS
egress to ECR and CloudWatch Logs public endpoints. Only port 443 is open.
```

- **GitHub**:
  - ruleset na `main`: tylko przez PR, wymagane checki `trivy` i `cost-gate`, blokada force push i usuwania,
  - środowisko `infra`: tylko gałąź `main`, jedyne źródło roli `gh-infra-apply`,
  - środowisko `infra-cost-approval`: wymagany reviewer,
  - akcje przypięte do pełnego SHA commita,
  - Dependabot (pip, docker, github-actions, terraform), co tydzień,
  - zmienne `AWS_ACCOUNT_ID`, `AWS_REGION`, `COST_THRESHOLD_USD`, jedyny sekret `INFRACOST_API_KEY` (darmowy klucz Infracost, bez dostępu do AWS),

- wynik nieudanych checków (pytest, Trivy, Terraform) jako komentarz na PR, aktualizowany przy kolejnych uruchomieniach.

5. Wdrożenie aplikacji (pipeline)

`app.yml`

- Wyzwalacze: push na `main` (zmiany w `app/`), PR (bez wdrożenia), ręcznie z wyborem `failure_mode`.
- Kroki: pytest -> build -> Trivy (CRITICAL kończy pipeline) -> SBOM CycloneDX (artefakt) -> [tylko main] OIDC -> push do ECR -> digest -> cosign sign (keyless) -> cosign attest SBOM -> `deploy.sh`.
- Tag obrazu: `<wersja>-<krótki sha>`, przy ręcznym uruchomieniu z dopiskiem `-<tryb>-<nr runu>` (tagi niezmienne, więc każdy build ma unikalny tag).
- Wdrożenie zawsze po digeście (`repo@sha256:...`), nie po tagu.

List. Podpis i atestacja SBOM (`.github/workflows/app.yml`, fragment)

```
- name: Sign image and attest SBOM (keyless)
  if: github.event_name != 'pull_request'
  env:
    IMAGE: ${{ steps.push.outputs.image }}
  run: |
    cosign sign --yes "$IMAGE"
    cosign attest --yes --type cyclonedx --predicate sbom.cdx.json "$IMAGE"
```

List. Weryfikacja podpisu przed wdrożeniem (`.github/scripts/deploy.sh`, fragment)

```
if ! cosign verify "$IMAGE" \
  --certificate-identity "$CERT_IDENTITY" \
  --certificate-oidc-issuer "$OIDC_ISSUER" > /dev/null; then
  summary "REJECTED: no valid signature from $CERT_IDENTITY, service not updated"
  exit 1
fi
```

- `CERT_IDENTITY` = workflow `app.yml` na `main`: obraz podpisany z innej gałęzi lub innego workflow jest odrzucany.
- Brak natywnej weryfikacji podpisów w ECS, dlatego weryfikacja w pipeline tuż przed `update-service`.
- Dalej `deploy.sh`: nowa rewizja task definition z digestem, `update-service`, śledzenie `describe-service-deployments`, raport SUCCESS / STOPPED / ROLLBACK.
- Etapy blue/green w ECS: SCALE_UP -> TEST_TRAFFIC_SHIFT -> PRODUCTION_TRAFFIC_SHIFT -> BAKE_TIME (5 min, stara wersja dalej działa) -> CLEAN_UP.

`infra.yml`

- PR: `plan` (fmt, validate, plan, plan.json, wycena kosztu, komentarz na PR), `trivy` (skan konfiguracji), `cost-gate`.
- Push na `main`: `apply` w środowisku `infra`.
- Wycena: Infracost CLI (open source, Apache 2.0), `infracost diff` na `plan.json`, komentarz z rozbiciem kosztu na PR, próg `COST_THRESHOLD_USD` decyduje o `cost-gate`.

List. Wybór środowiska w `cost-gate` (`.github/workflows/infra.yml`)

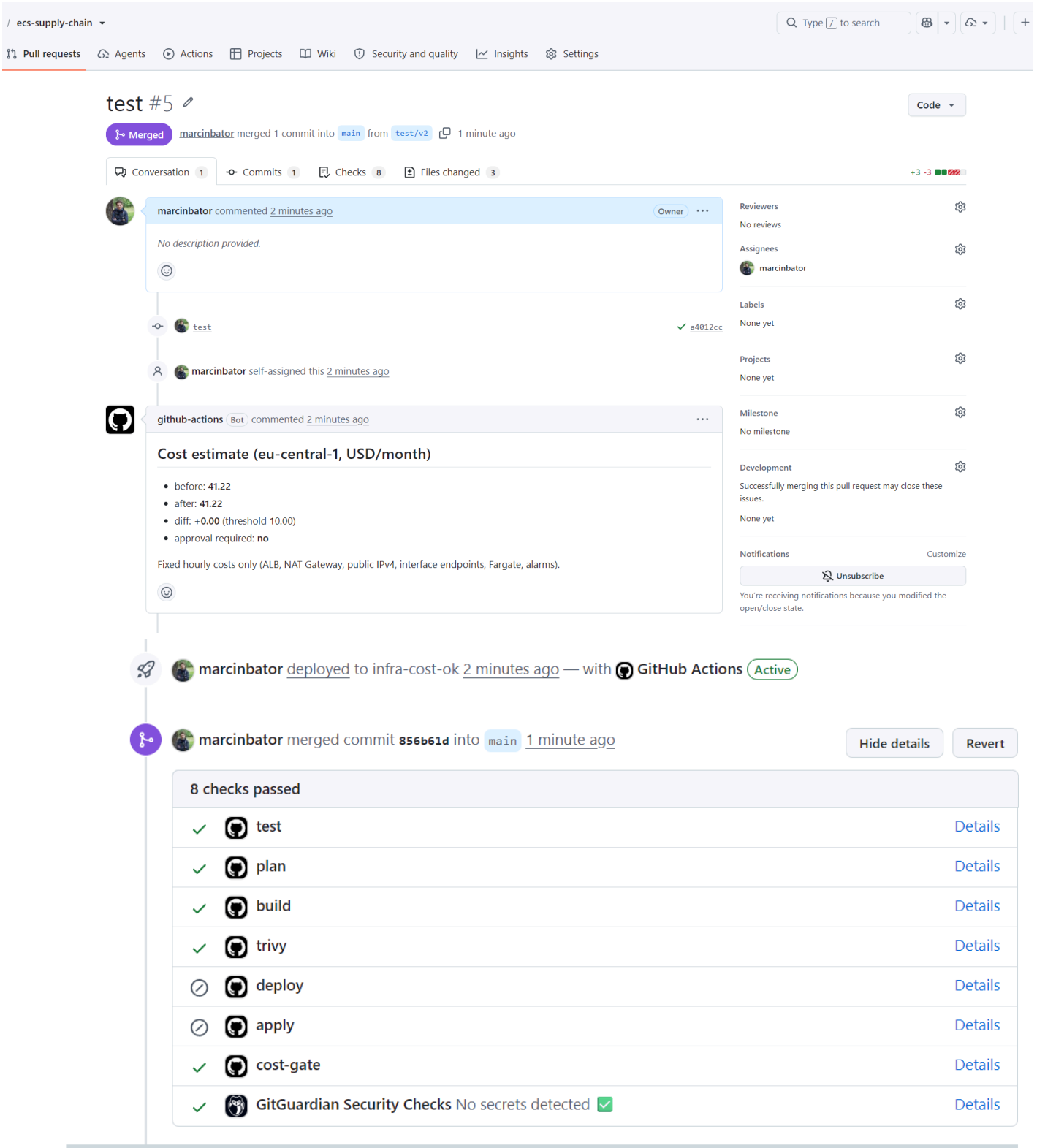
```
cost-gate:
  needs: plan
  runs-on: ubuntu-24.04
  environment: ${{ needs.plan.outputs.over_threshold == 'true' && 'infra-cost-approval' || 'infra-cost-ok' }}
```

6. Scenariusze

6.1 Poprawne wdrożenie (happy path)

- PR ze zmianą wersji, wycena kosztu +0.00, wszystkie checki zielone, merge.

- Pipeline: test, build, SBOM jako artefakt, push, podpis, wdrożenie.
- ECS: start green, przełączenie ruchu, bake time 5 min z alarmami OK, wyłączenie blue. Całość 7 min 41 s.
- ECR: obraz z niezmiennym tagiem + artefakty podpisu i atestacji (typ "Other"), skan przy push: 0 CRITICAL.
- Wynik: raport SUCCESS, zero błędów w `loadgen.sh`.


ecs-supply-chain
Type / to search
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
test #5
Code
Merged
marcinbator merged 1 commit into main from test/v2 1 minute ago
Conversation 1
Commits 1
Checks 8
Files changed 3
+3 -3
marcinbator commented 2 minutes ago
Owner
No description provided.
test
a4012cc
marcinbator self-assigned this 2 minutes ago
github-actions Bot commented 2 minutes ago
Cost estimate (eu-central-1, USD/month)
before: 41.22
after: 41.22
diff: +0.00 (threshold 10.00)
approval required: no
Fixed hourly costs only (ALB, NAT Gateway, public IPv4, interface endpoints, Fargate, alarms).
Reviewers
No reviews
Assignees
marcinbator
Labels
None yet
Projects
None yet
Milestone
No milestone
Development
Successfully merging this pull request may close these issues.
None yet
Notifications
Customize
Unsubscribe
You're receiving notifications because you modified the open/close state.
marcinbator deployed to infra-cost-ok 2 minutes ago — with GitHub Actions Active
marcinbator merged commit 856b61d into main 1 minute ago
Hide details
Revert
8 checks passed
test Details
plan Details
build Details
trivy Details
deploy Details
apply Details
cost-gate Details
GitGuardian Security Checks No secrets detected Details

Triggered via push 1 minute ago
marcinbator pushed 856b61d main

Status: In progress
Total duration: –
Artifacts: 1

app.yml
on: push

test 13s → build 1m 8s → deploy 4s

build summary

Pushed `104423791048.dkr.ecr.eu-central-1.amazonaws.com/ecs-supply-chain@sha256:734485aac127232e49d4926a672615de89a50727bb8d975366d4cea841e06328`

Job summary generated at run-time

Artifacts
Produced during runtime

Name	Size	Digest
sbom-2.0.0-856b61d	11.6 KB	sha256:69ebbf27ad4b9223bd93b8a06ba80fcb00069e028c3d7c1ac7b50c6cd4fa5…

Clusters > ecs-supply-chain > Services > ecs-supply-chain > Deployments

ecs-supply-chain Info

Last updated October 28, 2026, 18:27 (UTC+1:00) | 5 seconds | Delete service | Update service

Service overview Info

Status: Active
Tasks (1 Desired): 1 running | 1 pending
Task definition: revision: ecs-supply-chain:11
Deployment status: In progress

Health and metrics | Tasks | Logs | Deployments | Events | Configuration and networking - *updated* | Action logs | Service auto scaling | Event history | Tags

Latest service deployment

Deployment ID: 1JzSbKVPr929Vsu2lbEcZ

Send feedback | View action logs | Roll back

Deployment timeline Info

View in CloudTrail

Track the progress of each deployment phase.

- Deployment started
  Started at October 28, 2026, 18:26 (UTC+1:00)
- Scaling up green tasks
  Latest service events
  service has started 1 tasks: task 5483fd7df227436fabdc483b6fd5334b. — 6 seconds ago
- Validating deployment
- Launching green tasks
  New task launch progress
  0 running | 1 pending | 0 remaining
  0%
  0 of 1 tasks started
  Container health check: Health check not configured.
  Load balancer target health check: No targets
- Checking task health
- Test traffic shift
- Production traffic shift
- Bake time
- Terminating blue tasks
- Deployment complete

Monitoring
Real-time health signals for this deployment.
Circuit breaker: Monitoring
0/3 failed tasks
Alarms: Monitoring 0 2 0
Health check
Targets: Configured
VPC Lattice targets: Not configured
Containers: Not configured

Latest service deployment

Deployment ID: 1JzSbKVPr929Vsu2lbEcZ

Send feedback | View action logs | Roll back

Deployment timeline Info

View in CloudTrail

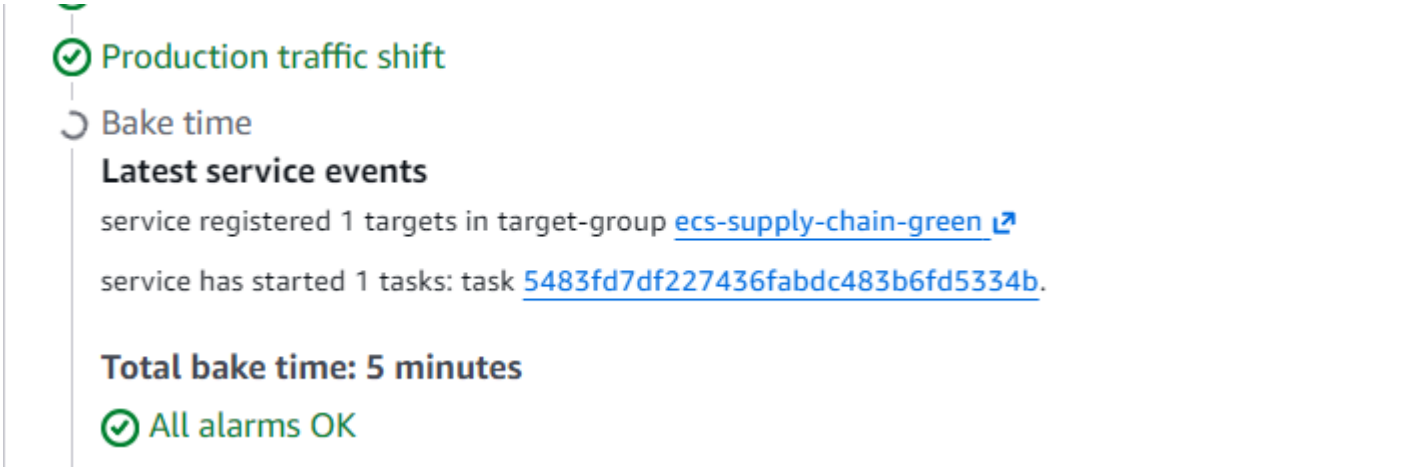
Track the progress of each deployment phase.

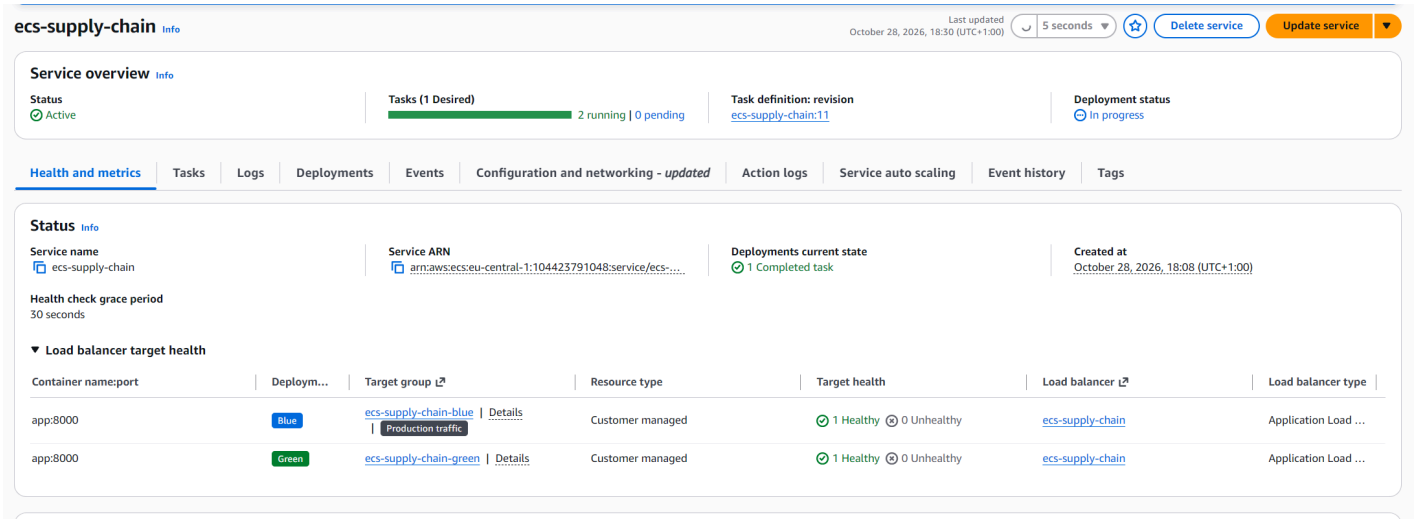
- Deployment started
  Started at October 28, 2026, 18:26 (UTC+1:00)
- Scaling up green tasks
  1 task started successfully
- Test traffic shift
- Production traffic shift
  Latest service events
  service registered 1 targets in target-group ecs-supply-chain-green — 1 minute ago
  service has started 1 tasks: task 5483fd7df227436fabdc483b6fd5334b. — 2 minutes ago
- Shifting production traffic
- Running post-shift checks
- Bake time
- Terminating blue tasks
- Deployment complete

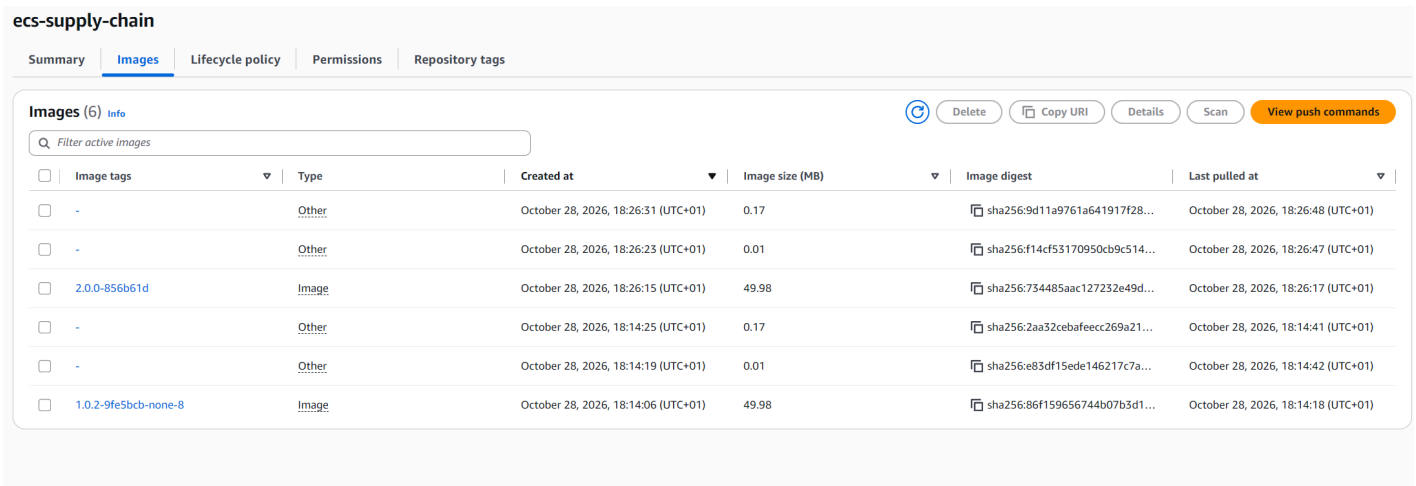
Monitoring
Real-time health signals for this deployment.
Circuit breaker: Monitoring complete
0/3 failed tasks
Alarms: Monitoring 0 2 0
Health check
Targets: Green 1 Healthy 0 Unhealthy
VPC Lattice targets: Not configured
Containers: Not configured

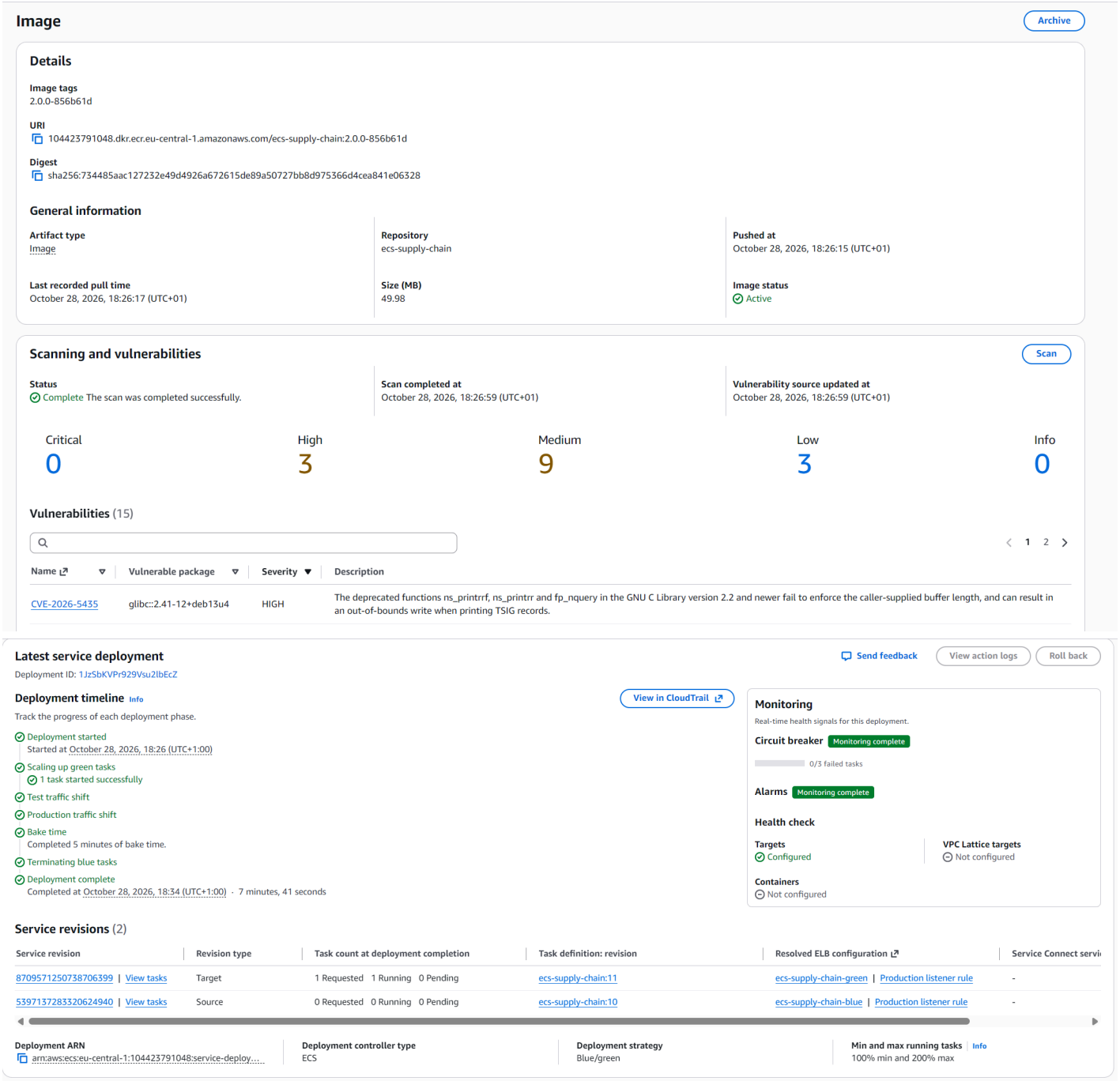
Service revisions (2)

Service revision	Revision type	Task count	Task definition: revision	Resolved ELB configuration	Service Connect servi
8709571250738706399 | View tasks	Target	1 Requested 1 Running 0 Pending	ecs-supply-chain:11	ecs-supply-chain-green | Production listener rule	-
5397137283320624940 | View tasks	Source	1 Requested 1 Running 0 Pending	ecs-supply-chain:10	ecs-supply-chain-blue | Production listener rule	-


Production traffic shift
Bake time
Latest service events
service registered 1 targets in target-group ecs-supply-chain-green
service has started 1 tasks: task 5483fd7df227436fabdc483b6fd5334b.
Total bake time: 5 minutes
All alarms OK


ecs-supply-chain Info
Last updated October 28, 2026, 18:30 (UTC+1:00)
5 seconds
Delete service
Update service
Service overview Info
Status
Active
Tasks (1 Desired)
2 running | 0 pending
Task definition: revision
ecs-supply-chain:11
Deployment status
In progress
Health and metrics
Tasks
Logs
Deployments
Events
Configuration and networking - updated
Action logs
Service auto scaling
Event history
Tags
Status Info
Service name
ecs-supply-chain
Service ARN
arn:aws:ecs:eu-central-1:104423791048:service/ecs-...
Deployments current state
1 Completed task
Created at
October 28, 2026, 18:08 (UTC+1:00)
Health check grace period
30 seconds
Load balancer target health
Container name:port
Deploym...
Target group
Resource type
Target health
Load balancer
Load balancer type
app:8000
Blue
ecs-supply-chain-blue | Details
Production traffic
Customer managed
1 Healthy 0 Unhealthy
ecs-supply-chain
Application Load ...
app:8000
Green
ecs-supply-chain-green | Details
Customer managed
1 Healthy 0 Unhealthy
ecs-supply-chain
Application Load ...


ecs-supply-chain
Summary
Images
Lifecycle policy
Permissions
Repository tags
Images (6) Info
Delete
Copy URI
Details
Scan
View push commands
Filter active images
Image tags
Type
Created at
Image size (MB)
Image digest
Last pulled at
- Other October 28, 2026, 18:26:31 (UTC+01) 0.17 sha256:9d11a9761a641917f28... October 28, 2026, 18:26:48 (UTC+01)
- Other October 28, 2026, 18:26:23 (UTC+01) 0.01 sha256:f14cf53170950cb9c514... October 28, 2026, 18:26:47 (UTC+01)
2.0.0-856b61d Image October 28, 2026, 18:26:15 (UTC+01) 49.98 sha256:734485aac127232e49d... October 28, 2026, 18:26:17 (UTC+01)
- Other October 28, 2026, 18:14:25 (UTC+01) 0.17 sha256:2aa32cebafeecc269a21... October 28, 2026, 18:14:41 (UTC+01)
- Other October 28, 2026, 18:14:19 (UTC+01) 0.01 sha256:e83df15ede146217c7a... October 28, 2026, 18:14:42 (UTC+01)
1.0.2-9fe5bcb-none-8 Image October 28, 2026, 18:14:06 (UTC+01) 49.98 sha256:86f159656744b07b3d1... October 28, 2026, 18:14:18 (UTC+01)



Image

Archive

Details

Image tags
2.0.0-856b61d

URI
104423791048.dkr.ecr.eu-central-1.amazonaws.com/ecs-supply-chain:2.0.0-856b61d

Digest
sha256:734485aac127232e49d4926a672615de89a50727bb8d975366d4cea841e06328

General information

Artifact type Image	**Repository** ecs-supply-chain	**Pushed at** October 28, 2026, 18:26:15 (UTC+01)
Last recorded pull time October 28, 2026, 18:26:17 (UTC+01)	**Size (MB)** 49.98	**Image status** Active

Scanning and vulnerabilities

Scan

Status Complete The scan was completed successfully.	**Scan completed at** October 28, 2026, 18:26:59 (UTC+01)	**Vulnerability source updated at** October 28, 2026, 18:26:59 (UTC+01)

Critical	High	Medium	Low	Info
0	3	9	3	0

Vulnerabilities (15)

1 2

Name	Vulnerable package	Severity	Description
CVE-2026-5435	glibc::2.41-12+deb13u4	HIGH	The deprecated functions ns_printrrf, ns_printrr and fp_nquery in the GNU C Library version 2.2 and newer fail to enforce the caller-supplied buffer length, and can result in an out-of-bounds write when printing TSIG records.

Latest service deployment

Send feedback | View action logs | Roll back

Deployment ID: 1JzSbKVPr929Vsu2lbEcZ

Deployment timeline Info

View in CloudTrail

Track the progress of each deployment phase.

- Deployment started
  Started at October 28, 2026, 18:26 (UTC+1:00)
- Scaling up green tasks
  - 1 task started successfully
- Test traffic shift
- Production traffic shift
- Bake time
  Completed 5 minutes of bake time.
- Terminating blue tasks
- Deployment complete
  Completed at October 28, 2026, 18:34 (UTC+1:00) · 7 minutes, 41 seconds

Monitoring

Real-time health signals for this deployment.

Circuit breaker Monitoring complete
0/3 failed tasks

Alarms Monitoring complete

Health check

Targets Configured	**VPC Lattice targets** Not configured

Containers
Not configured

Service revisions (2)

Service revision	Revision type	Task count at deployment completion	Task definition: revision	Resolved ELB configuration	Service Connect servi
8709571250738706399 | View tasks	Target	1 Requested 1 Running 0 Pending	ecs-supply-chain:11	ecs-supply-chain-green | Production listener rule	-
5397137283320624940 | View tasks	Source	0 Requested 0 Running 0 Pending	ecs-supply-chain:10	ecs-supply-chain-blue | Production listener rule	-

Deployment ARN arn:aws:ecs:eu-central-1:104423791048:service-deploy...	**Deployment controller type** ECS	**Deployment strategy** Blue/green	**Min and max running tasks** Info 100% min and 200% max

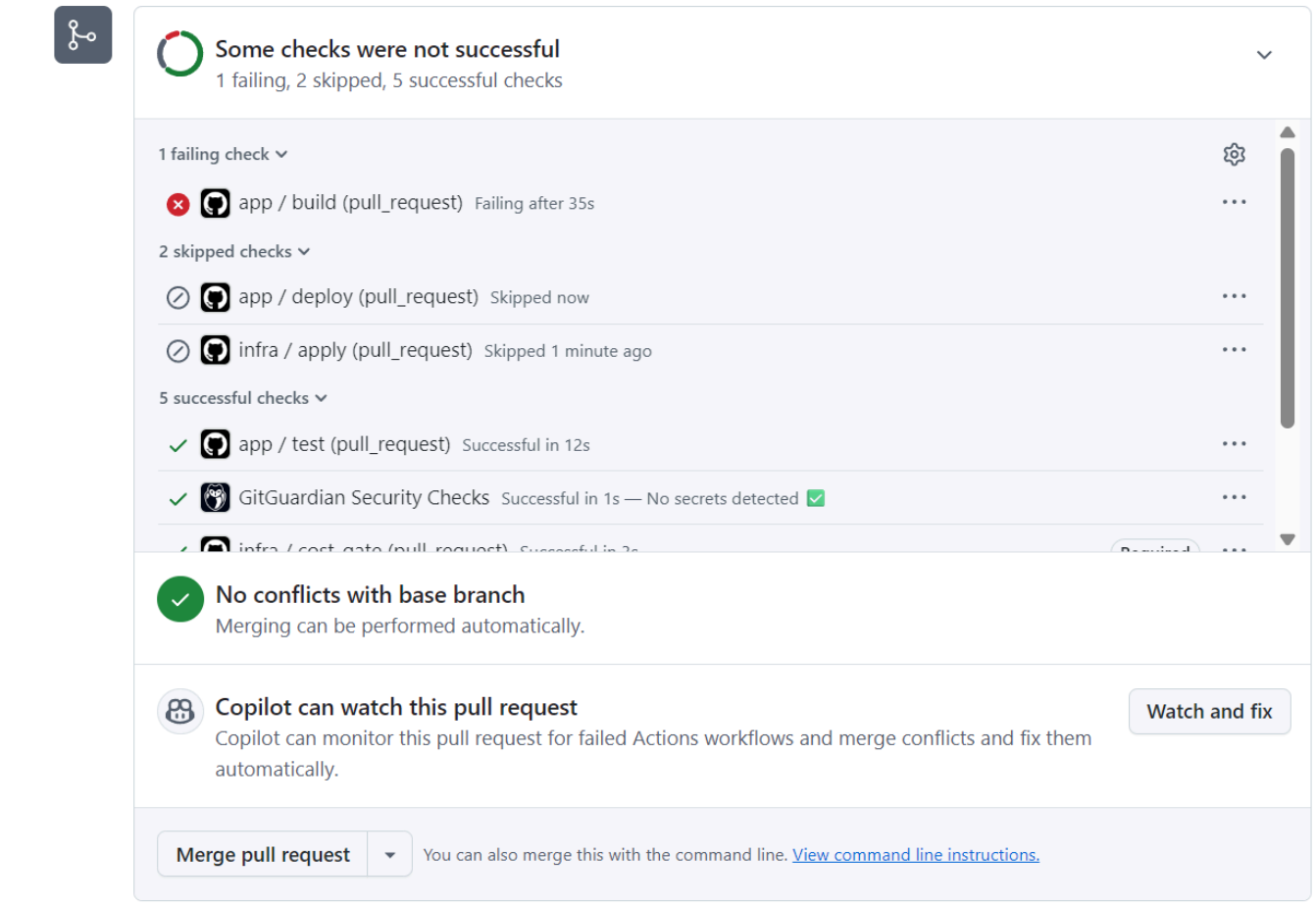
Verify signature and deploy (blue/green with bake time)

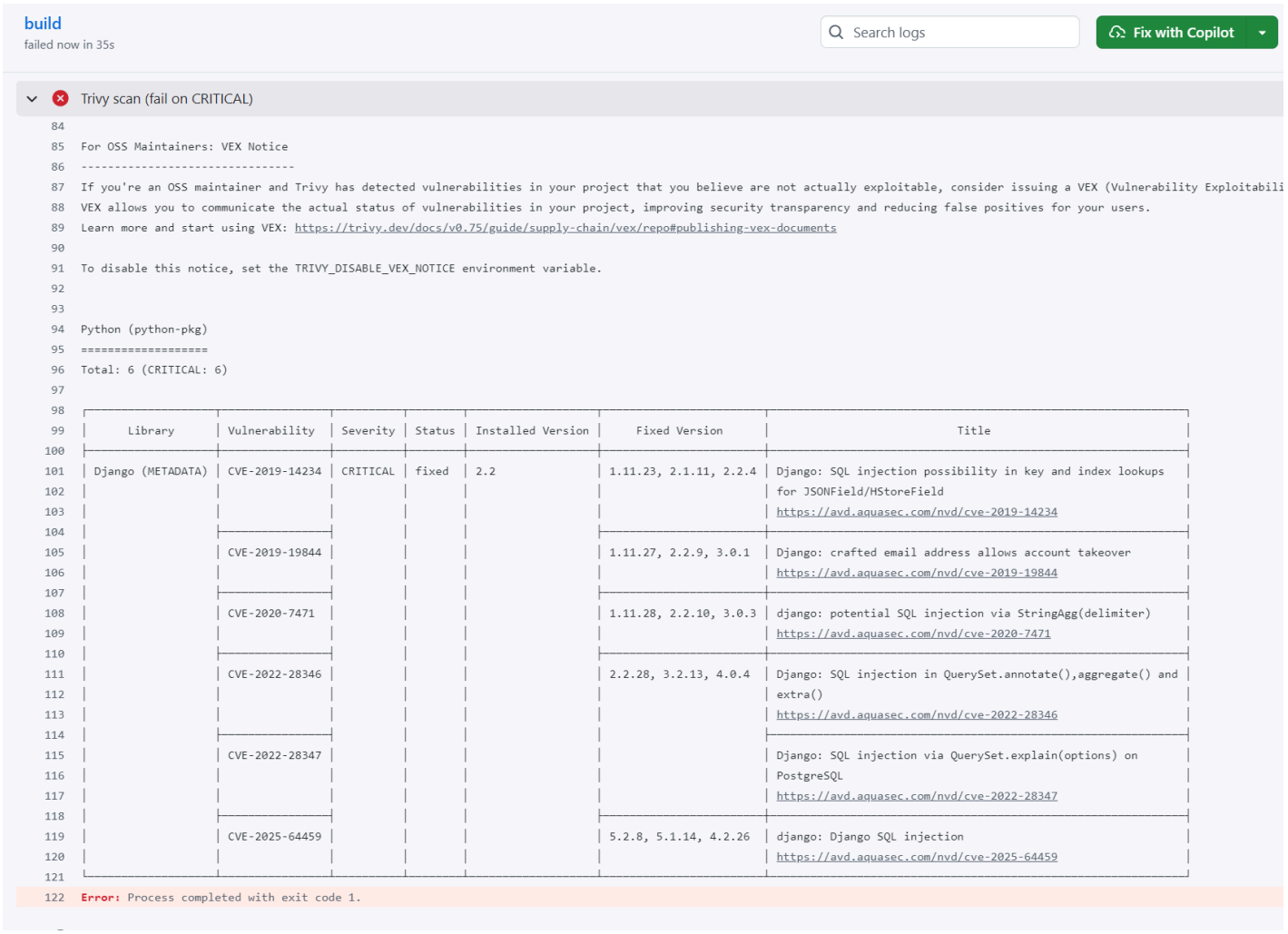
```
▶ Run scripts/deploy.sh "104423791048.dkr.ecr.eu-central-1.amazonaws.com/ecs-supply-chain@sha256:734485aac127232e49d4926a672615de89a50727bb8d975366d4cea841e06328"
Verifying signature of 104423791048.dkr.ecr.eu-central-1.amazonaws.com/ecs-supply-chain@sha256:734485aac127232e49d4926a672615de89a50727bb8d975366d4cea841e06328

Verification for 104423791048.dkr.ecr.eu-central-1.amazonaws.com/ecs-supply-chain@sha256:734485aac127232e49d4926a672615de89a50727bb8d975366d4cea841e06328 --
The following checks were performed on each of these signatures:
  - The cosign claims were validated
  - Existence of the claims in the transparency log was verified offline
  - The code-signing certificate was verified using trusted certificate authority certificates
Registered arn:aws:ecs:eu-central-1:104423791048:task-definition/ecs-supply-chain:11
Service deployment: arn:aws:ecs:eu-central-1:104423791048:service-deployment/ecs-supply-chain/ecs-supply-chain/1JzSbKVPr929Vsu2lbEcZ
17:27:02 status=IN_PROGRESS stage=PRE_SCALE_UP
17:27:23 status=IN_PROGRESS stage=SCALE_UP
17:27:45 status=IN_PROGRESS stage=SCALE_UP
17:28:06 status=IN_PROGRESS stage=TEST_TRAFFIC_SHIFT
17:28:27 status=IN_PROGRESS stage=PRODUCTION_TRAFFIC_SHIFT
17:28:48 status=IN_PROGRESS stage=PRODUCTION_TRAFFIC_SHIFT
17:29:09 status=IN_PROGRESS stage=BAKE_TIME
17:29:30 status=IN_PROGRESS stage=BAKE_TIME
17:29:51 status=IN_PROGRESS stage=BAKE_TIME
17:30:11 status=IN_PROGRESS stage=BAKE_TIME
17:30:32 status=IN_PROGRESS stage=BAKE_TIME
17:30:53 status=IN_PROGRESS stage=BAKE_TIME
17:31:14 status=IN_PROGRESS stage=BAKE_TIME
17:31:35 status=IN_PROGRESS stage=BAKE_TIME
17:31:56 status=IN_PROGRESS stage=BAKE_TIME
17:32:17 status=IN_PROGRESS stage=BAKE_TIME
17:32:38 status=IN_PROGRESS stage=BAKE_TIME
17:33:00 status=IN_PROGRESS stage=BAKE_TIME
17:33:21 status=IN_PROGRESS stage=BAKE_TIME
17:33:42 status=IN_PROGRESS stage=BAKE_TIME
17:34:03 status=IN_PROGRESS stage=CLEAN_UP
17:34:24 status=IN_PROGRESS stage=CLEAN_UP
17:34:45 status=SUCCESSFUL stage=-
SUCCESS: 104423791048.dkr.ecr.eu-central-1.amazonaws.com/ecs-supply-chain@sha256:734485aac127232e49d4926a672615de89a50727bb8d975366d4cea841e06328 deployed after bake time
```

6.2 Podatność CRITICAL

- PR dodaje `django==2.2` do `app/requirements.txt`.
- Trivy: 6 podatności CRITICAL (m.in. SQL injection), job `build` kończy się na kroku Trivy, obraz nie trafia do ECR.

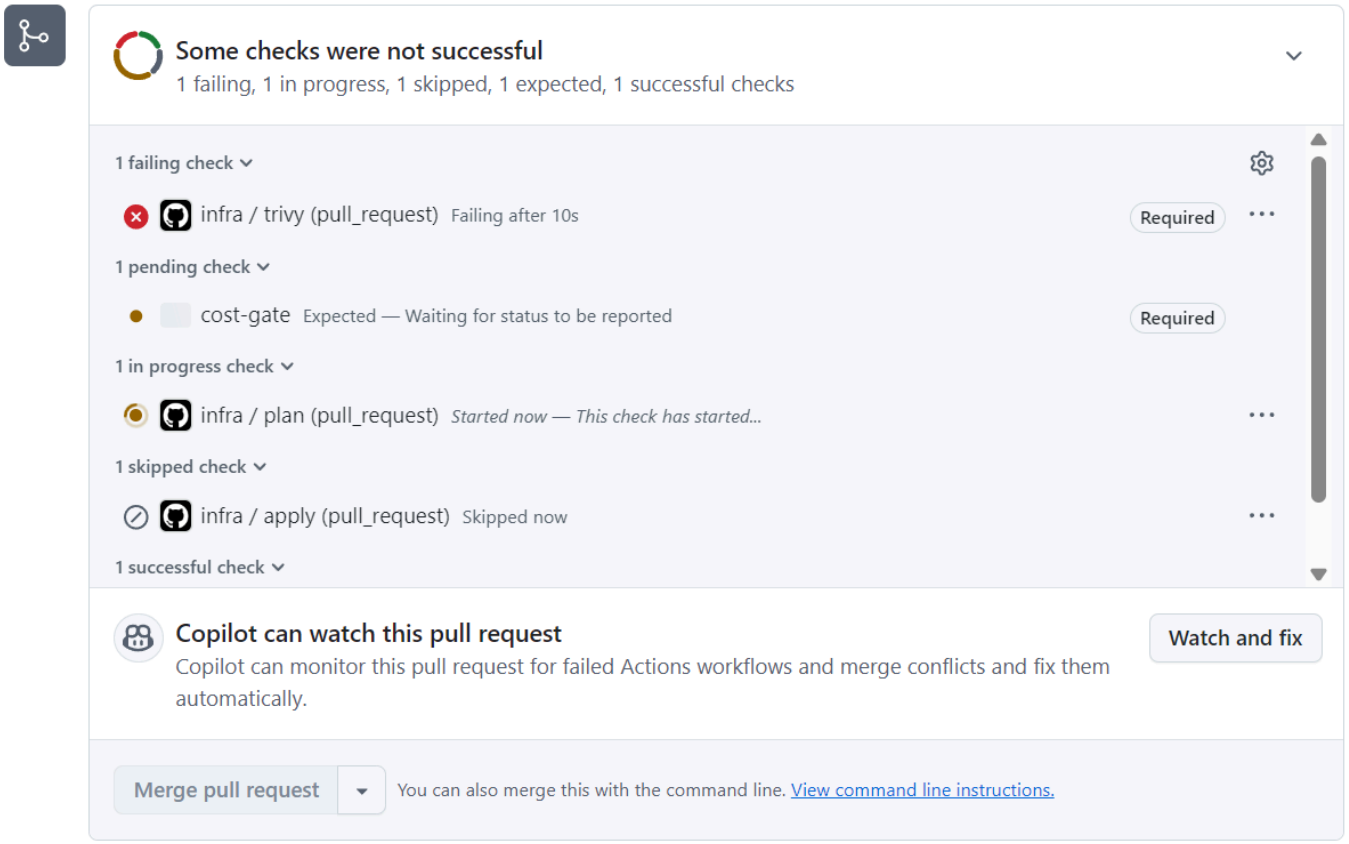




6.3 Security group otwarta na świat

- PR dodaje SG z portem 22 otwartym na `0.0.0.0/0`.

- `trivy config`: AWS-0107 (HIGH), check `trivy` wymagany, merge zablokowany.



Trivy config scan (fail on HIGH/CRITICAL)

```
network.tf (terraform)
======================
Tests: 1 (SUCCESSES: 0, FAILURES: 1)
Failures: 1 (HIGH: 1, CRITICAL: 0)

AWS-0107 (HIGH): Security group rule allows unrestricted ingress from any IP address.
========================================
Security groups provide stateful filtering of ingress and egress network traffic to AWS
resources. It is recommended that no security group allows unrestricted ingress access to
remote server administration ports, such as SSH to port 22 and RDP to port 3389.


See https://avd.aquasec.com/misconfig/aws-0107
----------------------------------------
 network.tf:49
   via network.tf:46-53 (aws_vpc_security_group_ingress_rule.ssh)
----------------------------------------
  46   resource "aws_vpc_security_group_ingress_rule" "ssh" {
  47     security_group_id = aws_security_group.ssh.id
  48     description       = "SSH from anywhere"
  49 [   cidr_ipv4         = "0.0.0.0/0"
  50     ip_protocol       = "tcp"
  51     from_port         = 22
  52     to_port           = 22
  53   }
----------------------------------------


Error: Process completed with exit code 1.
```

6.4 Kosztowna zmiana (NAT Gateway)

- PR dodaje NAT Gateway + EIP.
- Wycena Infracost: wzrost kosztu miesięcznego powyżej progu 10 USD, `cost-gate` czeka w `infra-cost-approval` na zatwierdzenie.



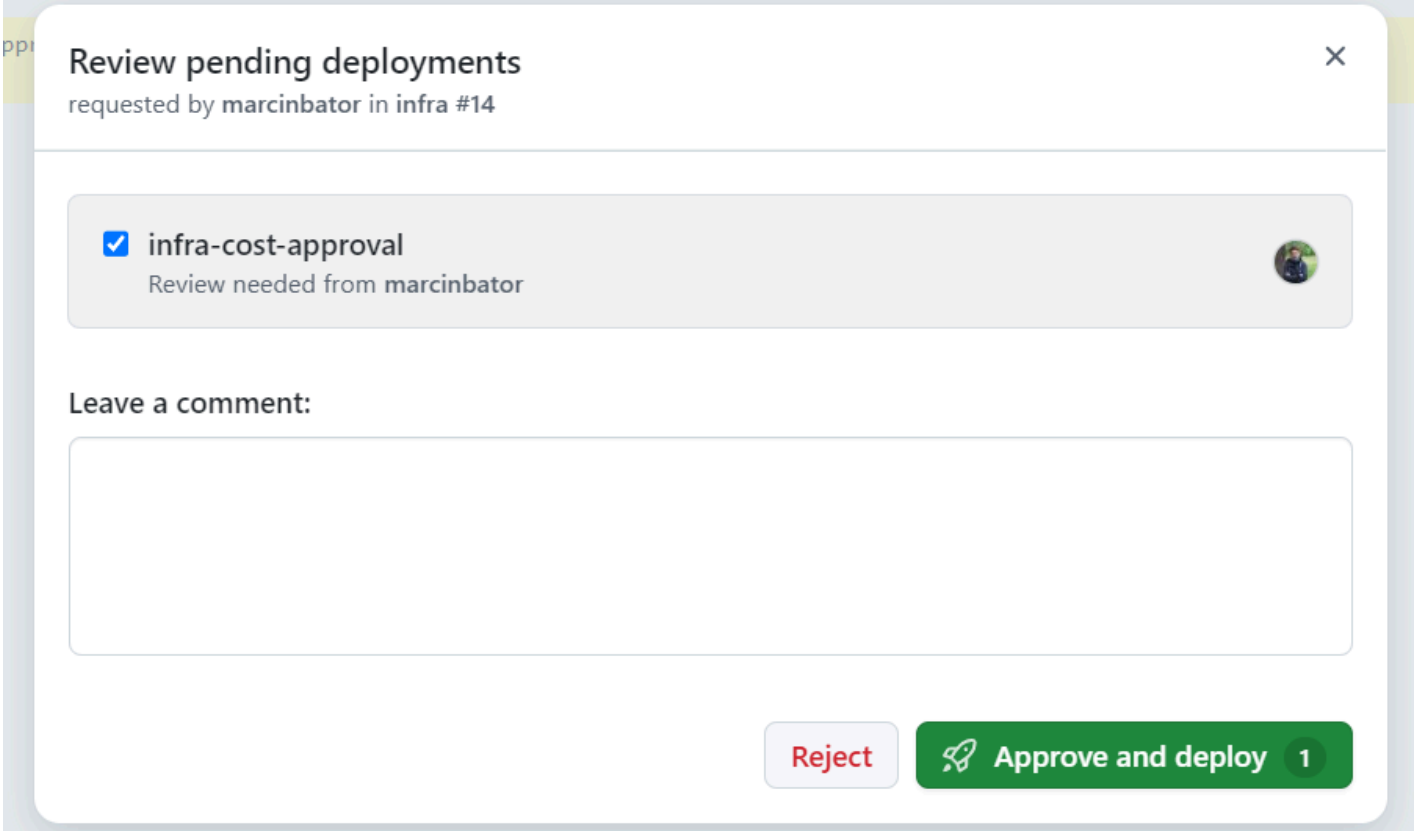
github-actions Bot commented 1 minute ago

Cost estimate (eu-central-1, USD/month)

- before: **41.22**
- after: **82.83**
- diff: **+41.61** (threshold 10.00)
- approval required: **yes**

Resource	Action	Diff
`aws_eip.nat`	create	+3.65
`aws_nat_gateway.demo`	create	+37.96

Fixed hourly costs only (ALB, NAT Gateway, public IPv4, interface endpoints, Fargate, alarms).



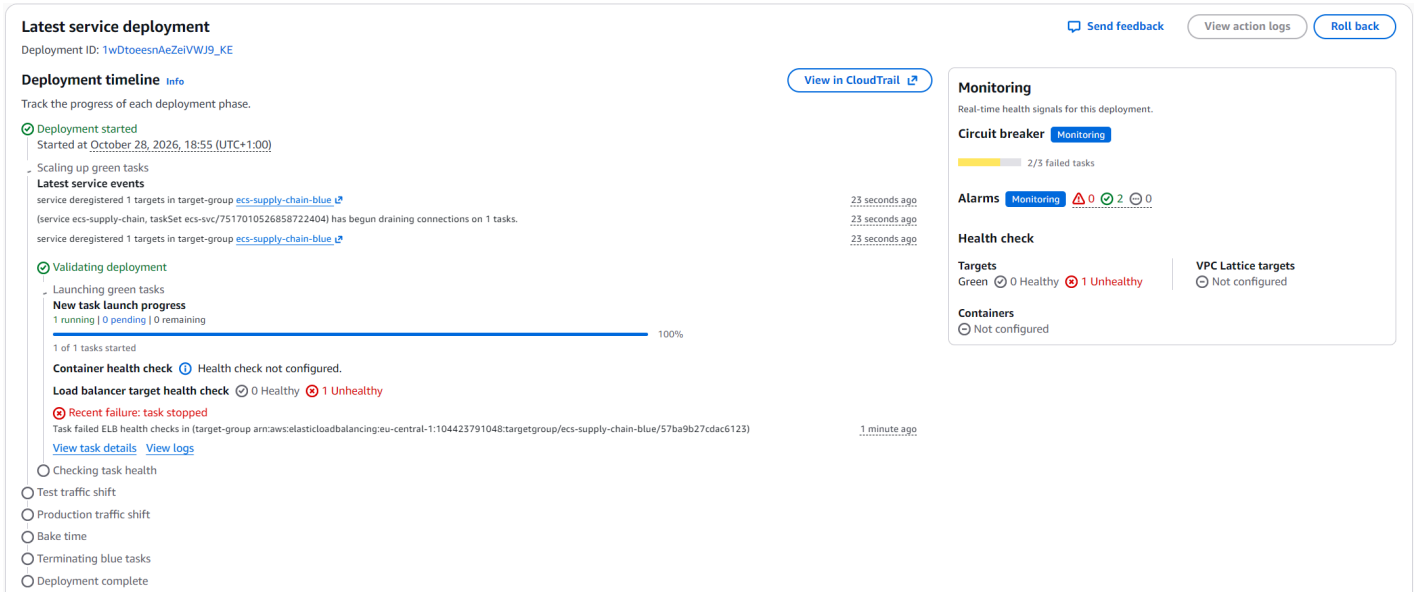
6.5 Niepodpisany obraz i nadpisanie tagu

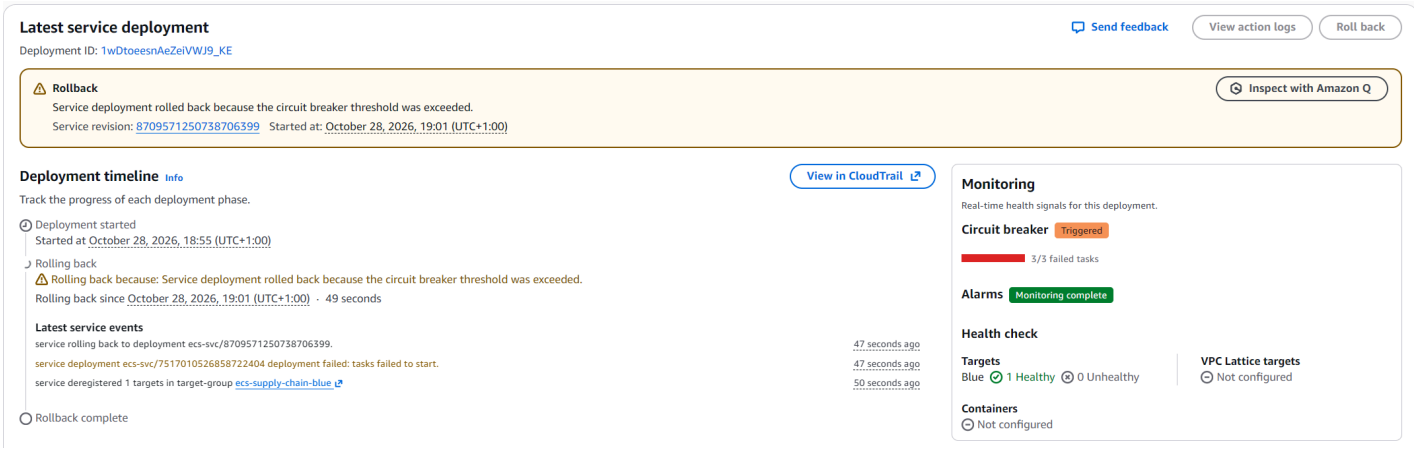
- `scripts/demo-unsigned.sh`: lokalny build, push do ECR z innym tagiem, próba wdrożenia tą samą ścieżką co pipeline (`deploy.sh`).
- `cosign verify` odrzuca obraz, usługa nie jest aktualizowana.
- Ponowny push pod ten sam tag: ECR odrzuca (`tag is immutable`).

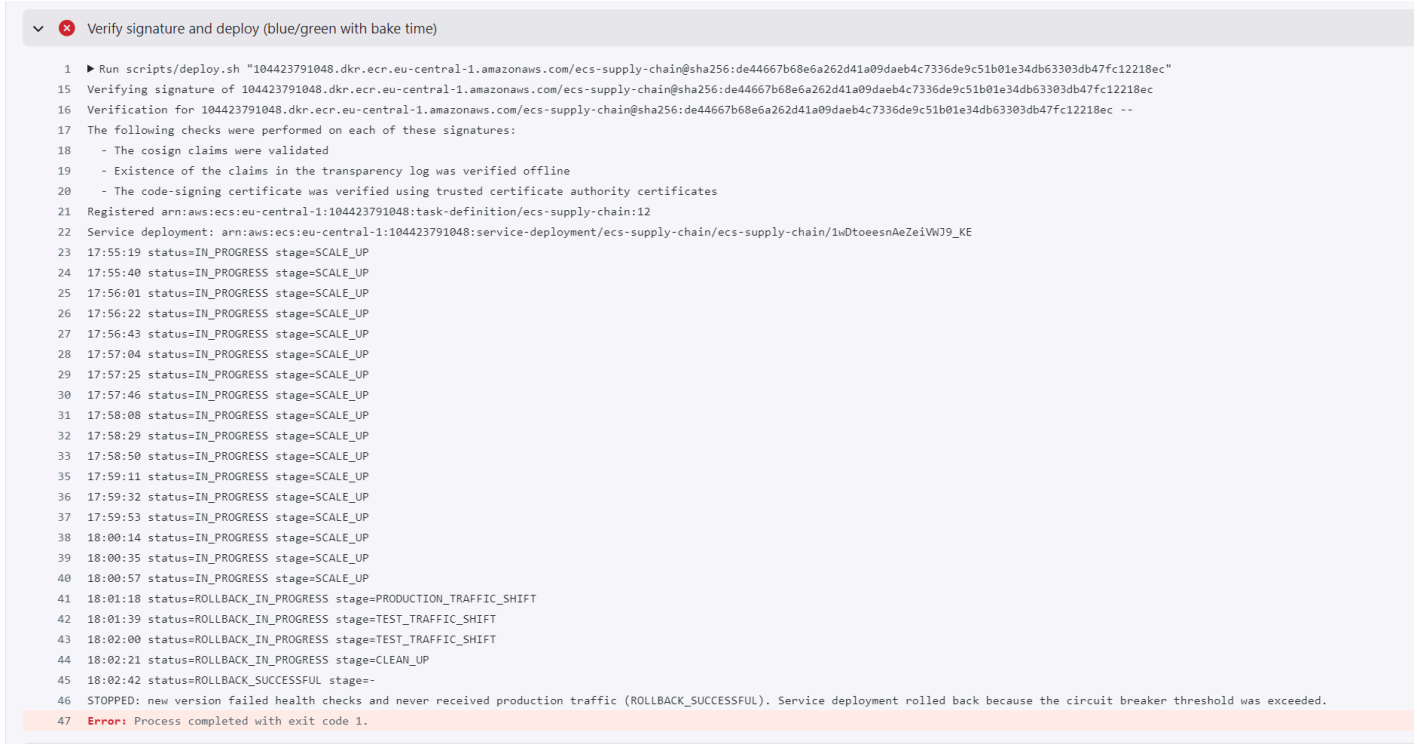
```
marcin@tufmb:~/studia/bezp_chmur/ecs-supply-chain$ scripts/demo-unsigned.sh
 => => unpacking to 104423791048.dkr.ecr.eu-central-1.amazonaws.com/ecs-supply-chain:unsigned-1793209820
The push refers to repository [104423791048.dkr.ecr.eu-central-1.amazonaws.com/ecs-supply-chain]
f2be67225471: Layer already exists
cb6a10e78a85: Pushed
44136fa355b3: Already exists
f5b253875fcb: Layer already exists
51c792a8ed8d: Pushed
01fe5efa5261: Pushed
ca30a39e3f58: Layer already exists
86586d55cc56: Layer already exists
0117bc864d6b: Layer already exists
1a46668363cf: Pushed
error from registry: The image tag 'unsigned-1793209820' already exists in the 'ecs-supply-chain' repository and cannot be overwritten because the tag is immutable.
OK: unsigned image rejected and tag overwrite refused by ECR
marcin@tufmb:~/studia/bezp_chmur/ecs-supply-chain$
```

6.6 Nowa wersja nie przechodzi healthchecku

- Run workflow z `failure_mode = health`.
- Zadanie green nie przechodzi healthchecku ALB, ruch produkcyjny cały czas na blue.
- Circuit breaker: 3/3 nieudane zadania, wdrożenie wycofane, raport STOPPED.
- To nie jest rollback w sensie celu 3: nowa wersja nigdy nie dostała ruchu.



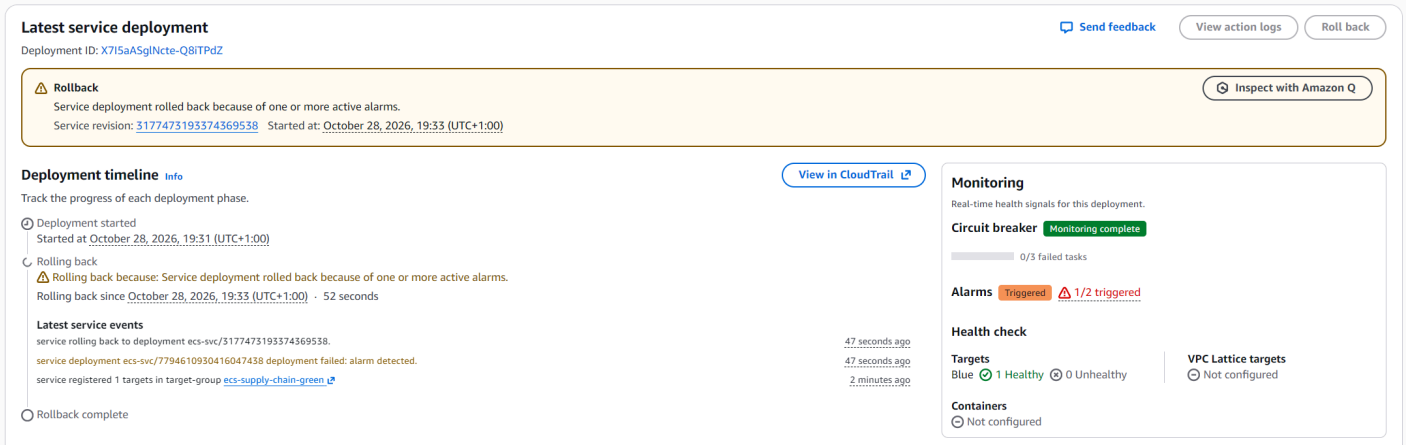




6.7 Rollback po alarmie w bake time

- Run workflow z `failure_mode = delayed`, w tle `loadgen.sh` (około 5 żądań/s).
- Nowa wersja `2.0.0-delayed` przejmuje ruch, alarm `target-5xx` przechodzi w ALARM, ECS przełącza ruch z powrotem, raport ROLLBACK.
- Wniosek: bez ruchu nie ma błędów, więc alarm nic nie wykryje (pierwsza próba bez `loadgen.sh` zakończyła się "udanym" wdrożeniem zepsutej wersji).

```
marcin@tufmb:~/studia/bezp_chmur/ecs-supply-chain$ scripts/loadgen.sh "$(terraform -chdir=terraform output -raw alb_url)"
19:34:15.145,500,
19:34:15.764,500,
19:34:16.340,500,
19:34:16.855,500,
19:34:17.369,500,
19:34:17.878,500,
19:34:18.335,500,
19:34:18.697,500,
19:34:19.170,500,
19:34:19.619,500,
19:34:20.025,500,
19:34:20.492,500,
19:34:20.949,500,
19:34:21.424,500,
```





Verify signature and deploy (blue/green with bake time)

```
▶ Run scripts/deploy.sh "104423791048.dkr.ecr.eu-central-1.amazonaws.com/ecs-supply-chain@sha256:541ad374b183f40a2e40c99756eb49b2567991d4ac62a8ef0dc17d4c3e752723"
Verifying signature of 104423791048.dkr.ecr.eu-central-1.amazonaws.com/ecs-supply-chain@sha256:541ad374b183f40a2e40c99756eb49b2567991d4ac62a8ef0dc17d4c3e752723
Verification for 104423791048.dkr.ecr.eu-central-1.amazonaws.com/ecs-supply-chain@sha256:541ad374b183f40a2e40c99756eb49b2567991d4ac62a8ef0dc17d4c3e752723 --
The following checks were performed on each of these signatures:
  - The cosign claims were validated
  - Existence of the claims in the transparency log was verified offline
  - The code-signing certificate was verified using trusted certificate authority certificates
Registered arn:aws:ecs:eu-central-1:104423791048:task-definition/ecs-supply-chain:14
Service deployment: arn:aws:ecs:eu-central-1:104423791048:service-deployment/ecs-supply-chain/ecs-supply-chain/X7I5aASglNcte-Q8iTPdZ
18:31:58 status=IN_PROGRESS stage=SCALE_UP
18:32:19 status=IN_PROGRESS stage=SCALE_UP
18:32:40 status=IN_PROGRESS stage=TEST_TRAFFIC_SHIFT
18:33:02 status=IN_PROGRESS stage=PRODUCTION_TRAFFIC_SHIFT
18:33:23 status=IN_PROGRESS stage=PRODUCTION_TRAFFIC_SHIFT
18:33:44 status=ROLLBACK_IN_PROGRESS stage=PRODUCTION_TRAFFIC_SHIFT
18:34:05 status=ROLLBACK_IN_PROGRESS stage=TEST_TRAFFIC_SHIFT
18:34:26 status=ROLLBACK_IN_PROGRESS stage=TEST_TRAFFIC_SHIFT
18:34:47 status=ROLLBACK_SUCCESSFUL stage=-
ROLLBACK: alarm (ecs-supply-chain-target-5xx) during bake time, traffic moved back to the previous version. Service deployment rolled back because of one or more active alarms.
Error: Process completed with exit code 1.
```

```
19:35:13.034,500,
19:35:13.591,500,
^Crequests: 668
errors: 608
error periods:
  19:29:26.936 - 19:33:15.612
  19:33:48.104 - 19:35:13.591
version changes:
  19:33:16.128: start -> 2.0.0-delayed
data: loadgen-20261028-192926.csv
marcin@tufmb:~/studia/bezp_chmur/ecs-supply-chain$
```

	time	code	version
480	19:33:40.495	200	2.0.0-delayed
481	19:33:40.788	200	2.0.0-delayed
482	19:33:41.415	200	2.0.0-delayed
483	19:33:41.716	200	2.0.0-delayed
484	19:33:42.138	200	2.0.0-delayed
485	19:33:42.663	200	2.0.0-delayed
486	19:33:43.168	200	2.0.0-delayed
487	19:33:43.680	200	2.0.0-delayed
488	19:33:44.193	200	2.0.0-delayed
489	19:33:44.897	200	2.0.0-delayed
490	19:33:45.511	200	2.0.0-delayed
491	19:33:46.025	200	2.0.0-delayed
492	19:33:46.458	200	2.0.0-delayed
493	19:33:46.937	200	2.0.0-delayed
494	19:33:47.536	200	2.0.0-delayed
495	19:33:48.104	500	
496	19:33:48.514	500	
497	19:33:49.236	500	
498	19:33:49.846	500	
499	19:33:50.324	500	
500	19:33:50.775	500	
501	19:33:51.247	500	
502	19:33:51.864	500	
503	19:33:52.423	500	
504	19:33:52.832	500	

